Assignment 3

- **Due** 8 Nov by 23:59
- **Points** 45
- **Submitting** a file upload

Assignment Name	Full-Stack Web Development for a Secure AI Application
Assignment Description	In this final assignment, you turn a trained model into a full-stack web application that people can use and that holds up when someone tries to misuse it. The front end must be built with React and the back end with FastAPI. A user provides input through the front end, the back end runs it through your model, and the result comes back to the user clearly and safely. Treat your model as a black box that takes an input and returns a prediction. You may use your Assignment 2 model, or any model you train for your topic. **This assignment does not mark your model's accuracy.** It marks how well you build, connect, and protect the application around the model, so a weaker model from Assignment 2 will not cost you marks here. Every topic in this unit is a cybersecurity problem, which means your application will receive hostile input by design. Making it secure is the focus of this assignment, and the main route to the highest marks. As the final assignment of the unit, this project carries the largest weight. It tests whether you can build a complete web application, connect a model to it, handle errors and hostile input safely, and explain clearly how all of it works.
Weight	45% of your total marks for the unit
Due Date	Sunday 8 November 2026, 11:59 pm (Week 13)
Submission	1. **Source code** (ZIP), including your README and your model file 2. **Video** (MP4 or AVI), 15 minutes maximum 3. **Meeting minutes** (PDF) 4. **Contribution form** (PDF) There is no written report. Section 3 describes each deliverable.
Late Penalties	Any submission after the due date counts as late. For the first 3 days late, no marks are deducted. From the fourth day, 5% of the available mark is deducted for each calendar day or part thereof.

	• Monday 9 to Wednesday 11 November, late, but no marks deducted • Thursday 12 November, 5% deducted • Friday 13 November, 10% deducted • Saturday 14 November, 15% deducted • Sunday 15 November, 20% deducted Submissions more than 7 calendar days late are not accepted. If serious circumstances mean you need more time than this, email your tutor and Lecturer as soon as possible to request an extension.

1. Project Description

This project develops a complete machine learning solution to a real cybersecurity problem, combining project management, design and technical implementation. Your team works through three phases, planning the project in Assignment 1, building and evaluating a model in Assignment 2, and delivering it as a secure web application in this assignment.

Your topic is the one your team has worked on all semester: one of prompt injection guardrail, AI-generated content detection, phishing URL detection, and log anomaly detection.

2. Overview of the tasks

Sections 2.1 to 2.5 apply to every topic. Together they describe a complete, working application.

2.1 Front end (React.js)

The front end is what users and markers see first, so it must work well and look professional.

- An input form that validates input before anything is sent to the back end
- Results shown clearly with the model's confidence, and the result as the most prominent element on screen
- Visible loading, success and error states, so the user always knows what is happening
- Error pages, including a page for addresses that do not exist and a fallback page when something fails unexpectedly, so the user never sees a blank screen
- At least two visualisations, each answering a different question about the results, with clear titles, labels and legends, and tooltips at minimum
- A consistent visual design throughout, with one colour scheme, consistent typography and spacing, and a clear layout
- A responsive layout that works on both a desktop and a phone-sized screen
- Basic accessibility, with a label on every input, readable colour contrast, and every function usable by keyboard

For a topic that checks one input at a time, one visualisation can show the current result while another compares all the inputs checked in the current session. More charts do not earn more marks. Each one should help the user understand something the others do not.

2.2 Back end (FastAPI)

- Request and response models defined with Pydantic
- At least two HTTP methods, each used for a genuine purpose
- Correct HTTP status codes, and clear error messages when input is invalid
- Every error, including unexpected ones, returned in one consistent JSON format, without internal details such as stack traces
- The model loaded once when the server starts, not on every request

2.3 Security hygiene

These are the minimum expected of any application in a cybersecurity unit. None of them takes much work, and an application that misses them cannot reach the higher grades.

- User input is never rendered as HTML
- Text inputs and file uploads have a size limit, enforced on the back end
- CORS allows only your own front end, not every origin
- Passwords, keys and other secrets are kept out of your source code, for example in environment variables

React escapes text for you, so the first requirement holds as long as you never bypass it, for example with dangerouslySetInnerHTML to highlight words. React 19 and later also neutralise a javascript: link written into href, but earlier versions at most print a warning, and no version protects a URL passed to window.open or assigned to window.location.

2.4 Running the application

- The whole application starts from your README on a clean machine
- Dependency versions are pinned, so the application runs the same way for the marker as it does for you

2.5 Requirements for your topic

You decide what your application takes as input, whether that is text, a URL, an image, audio, a log file, or a combination, so that it suits the model you are deploying. What your topic decides is what the application must show at minimum.

Topic	**The application shows at minimum**
Prompt injection guardrail	Whether the input is allowed or blocked, with confidence
AI-generated content detection	Whether the content is AI-generated or human-made, with confidence

Phishing URL detection	Whether the input is phishing or legitimate, with confidence
Log anomaly detection	Each session with an anomaly score, an overall summary, and a detail view for any session the user selects

2.6 Threat model

Before you defend your application, decide what you are defending it against. Write a short threat model that answers three questions.

1. Who would want to attack your application?
2. What are they trying to achieve?
3. What can they actually do to it?

Your answers decide which defences matter most, and you should use them to justify the choices you make in section 2.7. Half a page in your README is enough, and you should summarise it in your video before demonstrating your defences.

Each topic suggests a natural starting point.

Topic	**A likely attacker, and what they want**
Phishing URL detection	A phisher who wants their link to pass as legitimate
Prompt injection guardrail	An attacker who wants an injection to get past the guardrail
AI-generated content detection	Someone who wants AI-generated content to be judged human-made
Log anomaly detection	An intruder who wants their activity hidden among normal logs, using log lines that may themselves carry malicious content

2.7 Innovation. Security defences

Innovation in this assignment means defending your application against the attacks your threat model identifies, and proving each defence works by attacking your own application.

One principle runs through all of this. **Front-end checks are for usability. Back-end checks are for security.** Attackers do not use your interface, so several of the attacks below can only be demonstrated by sending requests directly, with a tool such as curl or Postman.

Not every risk applies to every application. An application whose back end never makes outbound requests cannot suffer SSRF, and one without a database cannot suffer SQL injection. Part of your threat model is working out which risks your design actually exposes, and saying briefly why the others do not apply.

Each risk below has a level, which sets the most a defence against it can earn. A Basic defence can earn up to 2 marks, an Intermediate one up to 3, and an Advanced one up to 4. Levels reflect how much work a defence takes to build and demonstrate, not how serious the risk is.

Risk	Level	When it applies to your application	How to prove you have stopped it
Unsafe model loading	Basic	Whenever you load a model with joblib or pickle, both of which can execute code hidden in the file. It matters most when the model file comes from a download script or from someone else	Check the model file's hash when the server starts, and refuse to load it if the hash does not match
Command injection	Basic	Whenever your back end runs a system command, such as the whois tool. Pasting the input into a shell command lets input such as example.com; ls run commands of the attacker's choosing	Submit a crafted input with curl and show the extra command never runs, because the input is validated and passed as a list of arguments, never through a shell
Missing rate limiting	Basic	Every application. FastAPI sets no limit, so anyone can flood the API or probe how the model makes its decisions	Send rapid requests and show the limit is enforced
SQL injection	Intermediate	Once you store data in a database, such as a history of past checks. Building a query by pasting the input into the SQL text lets input such as ' OR '1'='1 change what the query does	Submit a crafted input with curl, show it is stored and returned as plain data, and show that every query passes input as a parameter or through an ORM such as SQLAlchemy
Unsafe file upload	Intermediate	Whenever you accept files. FastAPI does not limit upload size for you, and saving a file under the user's own filename lets ../ write outside the upload folder. A size limit alone is already required	Send an oversized file, a crafted filename, and a file whose contents do not match its type with curl, and show all three are rejected

		by section 2.3, so this defence must go further	
Misconfiguration and vulnerable dependencies	Intermediate	Every application. Security headers are not set by default, debug features are easy to leave on, and any library can carry a known vulnerability	Run pip-audit and npm audit, scan the running application with OWASP ZAP, then fix or justify each finding
Denial of service	Advanced	Every application, because running a model costs far more than sending a request. Slow inference inside an async def endpoint makes every other user wait until it finishes, and a small crafted input can cost far more than it looks, such as text that makes a regular expression backtrack, or a 118 KB image that takes 300 MB of memory once decoded	Send one deliberately expensive request and show that other requests are still answered promptly, then send many at once and show the application rejects the excess with a 503 error instead of freezing
Server-side request forgery (SSRF)	Advanced	Only if your back end makes a request based on the input, such as fetching a page, looking up a domain, or downloading an image from a link	Submit inputs that point at internal addresses, such as 127.0.0.1 or a link that redirects there, and show from your back-end logs that no such request is made
Broken access control	Advanced	Only once you add user accounts. User B may then fetch user A's records by requesting them directly	Logged in as user B, request one of user A's record IDs with curl, and show the request is refused

A distributed denial-of-service attack, where many machines flood your network at once, is stopped upstream by a hosting provider or a content delivery network rather than by application code, and it cannot be tested fairly on a single laptop. What your back end can and should defend against is a request that costs it far more to handle than it costs an attacker to send.

Each topic has a natural place to start. SQL injection suits prompt injection, because every input is itself an attack string and is easily stored in a history of past checks. SSRF and command injection suit phishing, where lookups such as WHOIS are tempting to add. File upload suits any application that accepts files, such as log files, images or audio. Rate limiting suits AI-generated content detection, where people may probe the detector repeatedly. Denial of service suits any application whose inference is slow, such as one using a deep learning model or parsing large log files. Denial of service and misconfiguration apply to every application, so every topic has a route to full marks.

Choose up to two defences, from risks your application actually has, and name them in your README. Only those two are marked, so building more earns nothing extra. Your two defences together are capped at 8 marks, so full marks for this section need one Advanced defence together with an Intermediate or Advanced one. You may choose two Advanced defences, but the total is still capped at 8 marks.

Each defence is marked on its quality, up to the value of its level. The highest marks go to a defence that stops the attack and its obvious variations, is shown in your video with the attack succeeding without the defence and failing with it, is justified by your threat model, and has its cost explained. A defence that an obvious variation can get past, or that is shown working without the attack, earns less. A defence that does not appear in the video earns nothing.

If you choose both rate limiting and denial of service, each must rely on a different mechanism. A justified innovation of your own can take the place of one of the two, and your tutor will decide its level by the same standard. Security is still the recommended direction.

2.8 Advanced security

This part goes beyond typical undergraduate work and is aimed at groups who want the highest marks. These attacks exploit the fact that a trained model sits behind an API, however accurate the model is. Choose one, name it in your README, and do it properly. Only one is marked.

Side channels. A side channel leaks information through something other than the response itself, such as how long the response takes or how large it is. An application can therefore give its answer away even when the visible response reveals nothing.

Attack	**What it looks like in this application**	**What to measure**
Timing side channel in the decision	Every response looks the same, but some take longer, for example because blocked requests do extra work, so an attacker can infer the outcome from timing alone	The response-time distribution for each outcome, before and after you make response times uniform
Cache timing side channel	If you cache results to speed up repeated inputs, a cached answer returns far faster, so anyone can tell whether another user has already checked a given input. In our test, a cached answer took about 2 ms and a new one about 17 ms, with no overlap at all	Response times for inputs other users have submitted against new inputs, before and after you scope the cache to each user or remove the difference

Response size side channel	Responses that differ in size by outcome reveal the outcome even to someone who cannot read them. HTTPS hides the content of a response but not its length. In our test, an allowed response was 21 bytes and a blocked one 118 bytes	The response size for each outcome, before and after you return the same fields at the same size for every outcome
Login timing side channel	Only once you add user accounts. If the password check runs only for usernames that exist, a failed login reveals which usernames are real. In our test, a real username took about 490 ms to reject and an unknown one under 2 ms	Response times for real and unknown usernames, before and after you always run the password check, against a dummy hash when the username does not exist

Some side channels are created by defences elsewhere in this specification. A cache that protects you against denial of service is exactly what creates the cache timing side channel, so if you add one, check whether it leaks.

Attacks on the model through its API.

Attack	**What it looks like in this application**	**What to measure**
Membership inference	The model's confidence reveals whether a particular example was in its training data	Attack accuracy compared with the 50% of random guessing, before and after your defence
Model extraction	Repeated queries let an attacker train a copy of your model	How closely the copy agrees with your model as the number of queries grows, with and without your defence
Detection evasion	Small changes, such as swapping letters for look-alike Unicode characters, change what the model sees without changing what a person sees	The evasion success rate, before and after you normalise input before it reaches the model

Four conditions apply to this part.

- **Measure before and after.** Implement the attack, measure it, add your defence, and measure again. A claim without numbers earns no marks, because the obvious defence does not always work. Hiding the model's confidence and returning only a label, for example, does far less to stop model extraction than you might expect.
- **Secure the basics first.** This part only counts once your application meets the security hygiene requirements in section 2.3.
- **Explain what the defence costs.** Uniform response times slow the application down, hiding confidence gives the user less information, and a cache scoped to each user saves less work. Say what you gave up and why it was worth it.
- **Attack only your own application, on your own machine.**

Suggested reading

- OWASP Top 10Links to an external site., the standard list of the most critical web application security risks
- Bortz, A. and Boneh, D. (2007) 'Exposing private information by timing web applicationsLinks to an external site.', in *Proceedings of the 16th International Conference on World Wide Web*. ACM, pp. 621–628.
- Chen, S., Wang, R., Wang, X. and Zhang, K. (2010) 'Side-channel leaks in web applications: a reality today, a challenge tomorrowLinks to an external site.', in *2010 IEEE Symposium on Security and Privacy*. IEEE, pp. 191–206.
- Shokri, R., Stronati, M., Song, C. and Shmatikov, V. (2017) 'Membership inference attacks against machine learning modelsLinks to an external site.', in *2017 IEEE Symposium on Security and Privacy*. IEEE, pp. 3–18.
- Tramèr, F., Zhang, F., Juels, A., Reiter, M.K. and Ristenpart, T. (2016) 'Stealing machine learning models via prediction APIsLinks to an external site.', in *Proceedings of the 25th USENIX Security Symposium*. USENIX Association, pp. 601–618.

3 Deliverables

There is no written report. Everything a report would have covered now lives in your README, your API documentation, and your video.

1. **Source code (ZIP).** Front end and back end in separate folders. Include your model file, or a download script if it is too large to upload. Leave out node_modules.
2. **README.** Setup and run instructions, an architecture diagram, your pinned dependency versions, your threat model, references in Harvard style, a statement of how you used generative AI, and the defences and advanced attack you want marked.
3. **API documentation.** Use the /docs page that FastAPI generates for you. Make sure every endpoint has a description and an example request and response.
4. **Video, 15 minutes maximum.** See section 3.1.
5. **Meeting minutes (PDF).** At least one meeting each week since your team formed. You can use the attached meeting minutes as they are or as a reference to create your own. **Meeting Minutes Example.docx**
6. **Contribution form (PDF).** Download and complete Group Assessment Contribution Form.docx. You can download the form Group Assessment Contribution Form.docxDownload Group Assessment Contribution Form.docx

3.1 Video presentation

Your video is the main way your work is assessed, so plan it as carefully as the application itself. You decide how to structure it and how long to spend on each part, within the 15-minute maximum. Whatever structure you choose, the video must clearly explain the following.

- Who the application is for, and what problem it solves
- The full workflow from input to result, demonstrated live, including invalid input, error pages, and the phone-sized layout
- How the parts of your code connect, following one request from the front end through your API and model and back, shown in your actual code and in /docs
- How the key parts are implemented, walking through the code itself rather than only slides, and explaining in your own words what it does and why you wrote it that way
- The key design decisions you made, and why you made them
- Your threat model, how the application handles malicious input, and for any attack or defence you built, how the attack works and how your code stops it, demonstrated by attacking your own application
- What does not work yet, and what you would do next

Anything you want marked must appear in the video, because markers assess your work mainly from it.

The video is also marked in its own right, for how well you show that you understand your own work and why it is built the way it is. If you cannot explain part of your application, you lose marks here even if that part works. A missing feature loses marks only under the application criteria, not here as well.

- Your team decides who presents. Not every member has to appear in the video.
- If your file exceeds the Canvas upload limit, upload it to YouTube as an unlisted video, submit the link in a text file named group-name-video-link.txt, and keep the link active until marking is complete.
- If the video cannot be opened at the time of marking, most components of this assignment will receive zero. Check that it plays before you submit.